

MÃO NA MASSA

O Problema: O Formulário "Spaghetti"

O Cenário: A nossa loja virtual precisa de um painel de administração onde o gerente possa cadastrar novos produtos. O desenvolvedor anterior criou a tela, mas colocou absolutamente **todas** as regras dentro de um único arquivo de script. Ele lê o HTML, faz contas matemáticas, atualiza a lista e mexe no DOM, tudo ao mesmo tempo.

Sua Missão: Refatorar o código, modularizando, testando e ao final apresentando por escrito suas conclusões.

O Código Antigo (Tudo Misturado)

Arquivo: index.html (com o script embutido)

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Cadastro de Produtos - Caos</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    .form-group { margin-bottom: 10px; }
    .error { color: red; font-size: 0.9em; }
  </style>
</head>
<body>
  <h1>Cadastrar Novo Produto</h1>

  <div class="form-group">
    <label>Nome do Produto:</label>
    <input type="text" id="input-nome" placeholder="Ex: Teclado">
  </div>
  <div class="form-group">
    <label>Preço (R$):</label>
    <input type="number" id="input-preco" placeholder="Ex: 150.00">
  </div>
  <div class="form-group">
    <label>Quantidade:</label>
    <input type="number" id="input-qtd" placeholder="Ex: 2">
  </div>
  <button id="btn-salvar">Salvar Produto</button>
  <p id="msg-erro" class="error"></p>

  <hr>

  <h2>Lista de Estoque</h2>
  <ul id="lista-produtos"></ul>
  <h3 id="valor-total">Total em Estoque: R$ 0,00</h3>

  <script>
```

```
// --- O CAOS: Tudo no mesmo lugar ---
```

```
// 1. "Banco de dados" global
```

```
let produtosDb = [];
```

```
document.getElementById('btn-salvar').addEventListener('click', () => {
```

```
  // 2. Lendo o DOM diretamente misturado com a regra de negócio
```

```
  const nome = document.getElementById('input-nome').value;
```

```
  const preco = parseFloat(document.getElementById('input-preco').value);
```

```
  const qtd = parseInt(document.getElementById('input-qtd').value);
```

```
  const msgErro = document.getElementById('msg-erro');
```

```
  // Validação simples
```

```
  if(!nome || isNaN(preco) || isNaN(qtd)) {
```

```
    msgErro.innerText = "Preencha todos os campos corretamente!";
```

```
    return;
```

```
  }
```

```
  msgErro.innerText = ""; // Limpa o erro
```

```
  // 3. Salvando no "Banco"
```

```
  produtosDb.push({ nome, preco, qtd });
```

```
  // 4. Limpando os inputs (manipulação de DOM)
```

```
  document.getElementById('input-nome').value = "";
```

```
  document.getElementById('input-preco').value = "";
```

```
  document.getElementById('input-qtd').value = "";
```

```
  // 5. Atualizando a Interface Visual (DOM) e fazendo cálculos juntos
```

```
  const listaEl = document.getElementById('lista-produtos');
```

```
  listaEl.innerHTML = ""; // Limpa a lista
```

```
  let totalEstoque = 0;
```

```
  produtosDb.forEach(prod => {
```

```
    // Misturando lógica matemática com HTML
```

```
    const subtotal = prod.preco * prod.qtd;
```

```
    totalEstoque += subtotal;
```

```
    listaEl.innerHTML += `<li>${prod.qtd}x ${prod.nome} - R$ ${prod.preco.toFixed(2)} (Sub: R$  
    ${subtotal.toFixed(2)})</li>`;
```

```
  });
```

```
  // 6. Atualizando o total com formatação manual
```

```
  document.getElementById('valor-total').innerText = `Total em Estoque: R$
```

```
  ${totalEstoque.toFixed(2).replace('.', ',')}`;
```

```
  });
```

```
</script>
```

```
</body>
```

```
</html>
```

Resultado Esperado: O sistema completo, modularizado, funcional e com todas conclusões a problemas encontrados no caminho descritos em um relatório que deve ser apresentado junto ao sistema para o Professor em aula **HOJE**.
